Rex — Tài liệu kỹ thuật hợp nhất

1. Tổng quan

Rex là bản nâng cấp Laya từ bộ phân loại phản xạ một lượt (System V1) thành tác tử điều tra thích ứng (System V2): tự điều tra giao dịch đáng ngờ trong không gian vector ẩn, quyết định chặn hay cho qua, rồi lập báo cáo điều tra cho admin và thông báo ngắn cho khách.

Bối cảnh. Laya là công cụ ra quyết định mã nguồn mở, không tự hồi quy, xử lý đầu vào bằng một lượt truyền xuôi. Nó nhanh nhưng hay kết luận vội khi thiếu thông tin.

Mục tiêu. Thêm khả năng suy ngẫm lặp, gọi công cụ ngoài qua MCP, đặt giả thuyết rồi loại trừ, và tự dừng hoặc chuyển cho con người khi rủi ro cao. Ngân sách độ trễ: dưới 150 ms theo bản phân tích Laya, dưới 300 ms cho một vụ có 3 lần tra cứu theo tài liệu Rex; cần benchmark để chốt.

Hai nửa, nối nhau bằng một queue:

- **Nửa quyết định (Rex):** mở vụ, điều tra, ra kết luận. Rex và lớp điều phối lặp nhiều vòng: Rex đề xuất, lớp điều phối gọi tool rồi nạp kết quả. Vụ khó chuyển sang người duyệt. Kết luận của Rex hay của người đều đi qua Decision Adapter rồi mới vào queue.
- **Nửa lập báo cáo (Pipeline):** dựng báo cáo điều tra từ vết (trace) của Rex cho admin, gửi thông báo template cho khách. Nửa này không bao giờ đổi quyết định, chỉ ghi lại và diễn giải.

Ba cam kết của cả hệ thống:

1. Báo cáo không bịa: mọi bước và con số đều có trong vết điều tra; báo cáo không nói ngược quyết định.
2. Mỗi quyết định sinh đúng một báo cáo, kể cả khi message bị giao trùng hay server chết giữa chừng.
3. Báo cáo đã sinh thì chắc chắn lên được dashboard và thông báo tới được khách, hoặc ít nhất có cảnh báo cho đội vận hành nếu không tới.

Phạm vi code. `laya/common.py` (`DecisionModel`, đóng gói đầu vào) và `laya/agent.py` (`Agent` chuyển thành `InvestigatorAgent`); mã nguồn tham chiếu của model ở `modeling_rex_investigator.py`.

2. Thuật ngữ

Thuật ngữ	Ý nghĩa
M (Bộ nhớ Chứng cứ)	Tập vector mã hóa hồ sơ và kết quả tra cứu; cố định trong một vòng, chỉ được nối thêm khi có kết quả tool mới. Nằm trong RAM, mất khi vụ xong
h (Vector Suy ngẫm / trạng thái)	Con trỏ trạng thái nhận thức 768 chiều, khởi tạo từ `[CLS]`, tích lũy qua các chu kỳ và được giữ qua các vòng ngoài
S (Sufficiency)	Xác suất đã đủ chứng cứ để được phép kết luận, thuộc [0, 1]. Không phải độ tự tin
R / risk	Xác suất tổn thất nếu phán quyết sai, thuộc [0, 1]; vượt ngưỡng thì chuyển người
Vòng trong	Suy ngẫm trong tensor, 2–4 chu kỳ, không gọi ra ngoài, luôn dừng
Vòng ngoài	Vòng điều tra: gọi tool, nạp chứng cứ mới, suy ngẫm tiếp; có giới hạn N_max
Lớp điều phối (Orchestrator)	Code bất đồng bộ bao quanh model: gọi MCP Server, giữ giới hạn an toàn, quyết định vụ kết thúc thế nào
State Store	Cơ sở dữ liệu có cấu trúc chứa hồ sơ vụ và `tool_results`; lưu lâu dài, người đọc được
Trace (vết điều tra)	Bản ghi từng bước Rex đã làm: tool, tham số, kết quả, S từng vòng; nguồn duy nhất cho báo cáo
POE	Process of Elimination: loại trừ giả thuyết trước khi phán quyết
Decision Adapter	Code không AI, ghép kết quả vụ với hồ sơ thành message vào queue
Lease	Cờ “đang xử lý” có hạn dùng (60 giây), chặn hai worker cùng sinh một báo cáo
Outbox	Bảng `report_outbox` ghi “chờ gửi” trong cùng transaction với báo cáo, để việc gửi không bị mất
HITL	Human-in-the-loop: người duyệt tiếp quản toàn bộ vụ, Rex dừng hẳn

3. Hiện trạng Laya (System v2)

Laya cho một phán quyết sau đúng một lượt truyền xuôi: nhanh, nhưng độ sâu tính toán bị khóa cứng và không có cơ chế tự kiểm tra.

Hạng mục	Hiện trạng
Backbone	ModernBERT-large (421M tham số, tiếng Anh, 28 tầng) hoặc mmBERT-base (322M, đa ngôn ngữ, 22 tầng)
Chú ý	Hai chiều (bidirectional), thấy toàn bộ chuỗi
Độ trễ	Khoảng 32,8 ms trên một GPU; 7,2 ms mỗi câu hỏi khi xử lý theo lô
Huấn luyện	RLCD: học tăng cường với quy tắc tính điểm có hiệu chuẩn
Đầu ra	Logit trực tiếp cho từng phương án, không sinh từ tự do

Đầu vào được `laya/common.py` ghép phẳng thành một chuỗi, mỗi phương án có một token `[MASK]` riêng và dùng chung ngân sách độ dài:

```
[CLS] <type> instructions [SEP] [MASK] opt0 [MASK] opt1 ... [SEP] state [SEP]
```

Tầng quyết định tại các vị trí `[MASK]` gồm LayerNorm, Linear, GELU, rồi Linear cuối. `act_head` đọc token `[CLS]`.

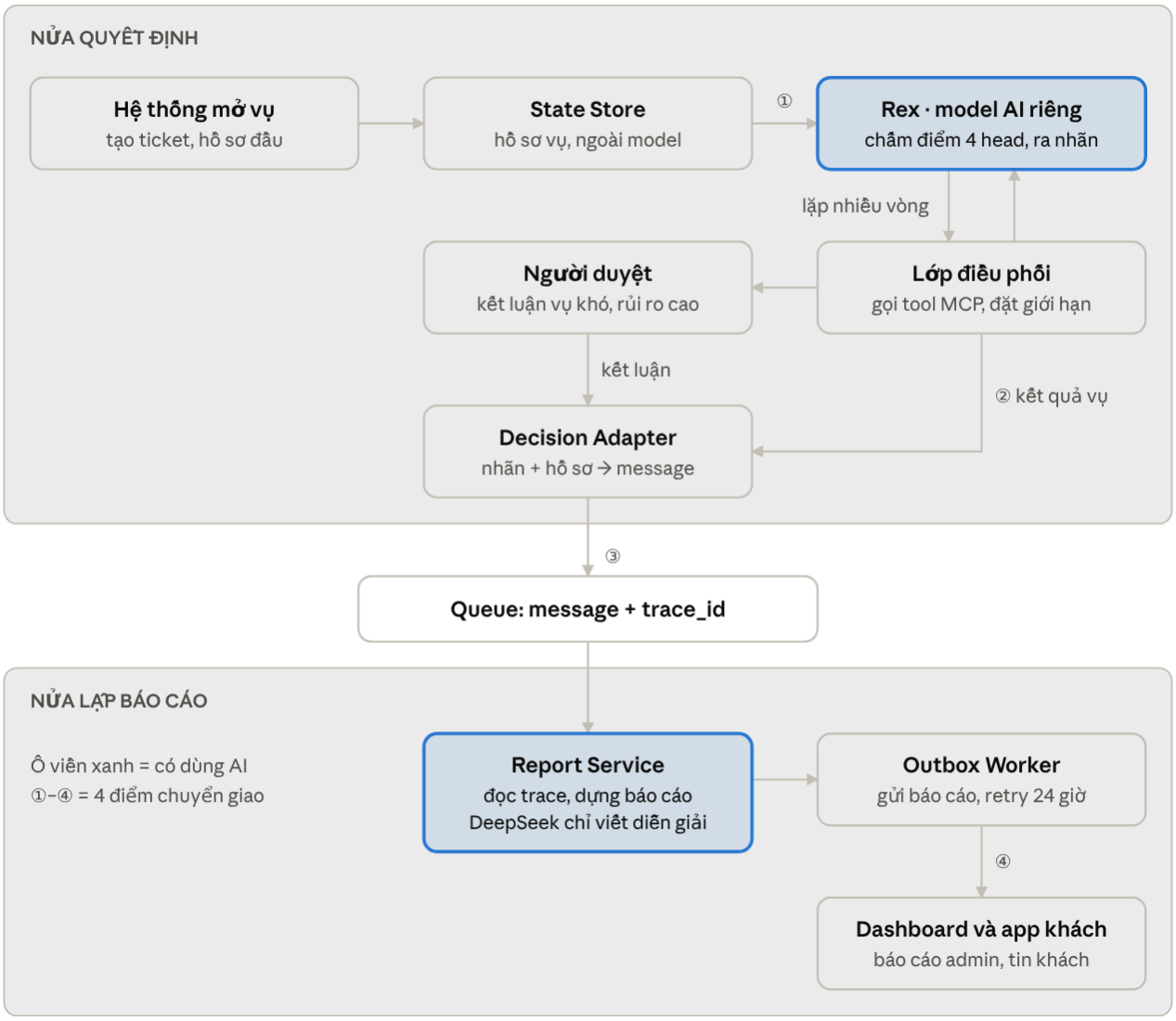
Giới hạn cần khắc phục:

- **Độ sâu cố định:** biểu diễn chỉ biến đổi một lần, không lặp để kiểm tra giả thuyết hay tích lũy chứng cứ nhiều bước.
- **Khớp mẫu bề mặt:** một đặc trưng nổi bật có thể kéo ra kết luận sai khi thông tin chưa đủ.
- **Nhiều lựa chọn:** hiệu năng tụt rõ khi số lựa chọn vượt 20.
- **Không tương tác môi trường:** xử lý một lần rồi kết thúc, không tra cứu dữ liệu bổ sung.

4. Kiến trúc tổng thể

Hệ thống có 8 khối chia hai nửa, chỉ nói chuyện với nhau qua 4 điểm chuyển giao; nửa quyết định chạy hai vòng lặp lồng nhau cho đến khi đủ chứng cứ hoặc phải chuyển người.

Mọi kết luận đi qua Decision Adapter và queue rồi mới thành báo cáo



toàn hệ thống · 8 khối, 2 nửa, 4 điểm chuyển giao

Rex và lớp điều phối lặp qua lại đến khi đủ chứng cứ; vụ khó sang người duyệt; nửa dưới chỉ ghi lại và diễn giải, không bao giờ đổi quyết định.

4.1 Bốn điểm chuyển giao (hợp đồng)

Mỗi điểm có một format cố định; giữ đúng hợp đồng thì sửa bên trong mỗi khối không ảnh hưởng bên kia.

#	Từ → Đến	Chuyển giao gì	Format	Đã định nghĩa chưa
1	Mở vụ → State Store → Rex	Hồ sơ vụ	Bản ghi State Store (mục 10.1)	Chưa, tài liệu Rex chỉ nhắc tên

#	Từ → Đến	Chuyển giao gì	Format	Đã định nghĩa chưa
2	Lớp điều phối → Adapter, kho trace	Kết quả vụ và vết điều tra	JSON kết quả vụ (10.2), trace (11.4)	Chưa, mới là đề xuất
3	Adapter → Queue → Report Service	Quyết định và `trace_id`	Message `Decision_Finalized` (11.1)	Rồi
4	Outbox Worker → Dashboard admin, app khách	Báo cáo điều tra và thông báo	JSON 6 phần (11.3), template (11.7)	Rồi, đến từng trường

4.2 Hai vòng lặp, đừng nhầm

- **Vòng trong (35–50 ms):** h đọc lại M qua cross-attention 2–4 lần, kiểm tra nhất quán và cập nhật S, R. Số bước cố định nên luôn dừng.
- **Vòng ngoài:** mỗi vòng một lượt gọi tool. Orchestrator chạy tool do Tool Matcher chọn, mã hóa kết quả, nối vào M rồi giữ nguyên h cho lượt sau. Có thể kéo dài nếu S mãi không đạt ngưỡng, nên lớp điều phối đặt giới hạn.

4.3 Điều kiện dừng

Chỉ có một đường dẫn tới nhãn kết luận: **S ≥ 0,85 và vụ không rủi ro cao**. Mọi đường còn lại hoặc quay về tra cứu, hoặc chuyển người duyệt khi: risk vượt ngưỡng R hoặc chạm sàn luật rủi ro; S không tăng quá ε sau hai vòng liên tiếp (bế tắc); hết N_max lần gọi tool; hoặc tool bị chặn (chi tiết ở mục 9).

4.4 Vì sao không dùng LLM agent kiểu ReAct

ReAct viết suy nghĩ ra chữ mỗi bước, mất 2.000–5.000 ms, dễ bịa thông tin và có thể trả JSON sai cú pháp. Rex mất 35–50 ms mỗi bước, đầu ra luôn thuộc một tập nhãn đóng (`CALL_TOOL_*` hoặc `VERDICT_*`), và chạy được trên CPU.

5. Đi theo một vụ từ đầu đến cuối

Vụ của anh A đi qua 9 bước và kết thúc bằng BLOCK sau 3 vòng Rex, 2 lần tra cứu. Điểm S, risk, xác suất là số minh họa.

Tình huống: 22:58, tài khoản anh A đăng nhập từ thiết bị lạ rồi đặt ngay lệnh rút 50 triệu.

Bước	Khối	Chuyện gì xảy ra	Kết quả
1	Hệ thống mở vụ	Cảnh báo bắt được “đăng nhập từ vị trí lạ rồi rút tiền ngay”	Mở vụ TCK-2026-88712, ghi hồ sơ vào State Store
2	Rex, vòng 1	Đọc hồ sơ, mới có thông tin giao dịch, chưa biết gì về IP hay thiết bị	S = 0,40, chưa đủ. Chọn tool tra IP
3	Lớp điều phối	Tra IP là tool chỉ đọc nên được chạy; nạp kết quả cho Rex, ghi trace	IP thuộc quốc gia khách chưa từng đăng nhập
4	Rex, vòng 2	Đọc lại hồ sơ kèm kết quả tra IP	S = 0,62, vẫn chưa đủ. Chọn tool tra thiết bị
5	Lớp điều phối	Chạy tool tra thiết bị, nạp kết quả, ghi trace	Thiết bị mới, gắn với 3 tài khoản khác
6	Rex, vòng 3	Đọc lại toàn bộ chứng cứ	S = 0,91 (đủ), risk = 0,30 (dưới ngưỡng), Verdict nghiêng FRAUD 0,88. Ra `VERDICT_FRAUD`
7	Decision Adapter	Ghép nhãn với hồ sơ, đổi FRAUD thành BLOCK, gắn `trace_id`	Message `final_action = BLOCK` vào queue
8	Report Service	Nhận message, giữ lease, đọc trace. Code điền dữ kiện, DeepSeek viết diễn giải, rồi kiểm tra không bịa	Báo cáo SUCCESS, kèm dòng “chờ gửi” trong outbox
9	Outbox Worker	Đẩy báo cáo lên dashboard, gửi thông báo template cho anh A	Admin đọc được báo cáo, anh A nhận thông báo

Thông báo anh A nhận (template, không dùng LLM): “Giao dịch rút tiền tạm thời bị chặn. Chúng tôi phát hiện hoạt động bất thường trên tài khoản và đã tạm dừng giao dịch để bảo vệ bạn. Vui lòng liên hệ CSKH với mã tra cứu TCK-2026-88712.”

Hai đầu ra dành cho hai người đọc khác nhau: admin cần đủ tín hiệu và từng bước Rex đã tra; khách chỉ cần biết chuyện gì xảy ra và làm gì tiếp, nên thông báo không nêu quốc gia, IP hay thiết bị.

6. Thiết kế model Rex

Mỗi vòng Rex chạy qua 3 tầng — đọc (Encoder), nghĩ (Latent PonderBlock), chấm điểm (4 head) — và không sinh chữ; kết quả chỉ là các con số.

6.1 Tầng đọc: tách ngữ cảnh khỏi trạng thái

- Hồ sơ được cắt thành token; Encoder (ModernBERT) đổi mỗi token thành vector 768 số mã hóa ý nghĩa trong ngữ cảnh.
- **Bộ nhớ Chứng cứ M = {m1, …, mK}:** toàn bộ vector token, mã hóa một lần, giữ cố định trong vòng.
- **Vector Suy ngẫm h_0 = h[CLS]:** điểm xuất phát, con trỏ nhận thức tích lũy qua từng chu kỳ.
- Khi tool trả kết quả, Encoder chỉ mã hóa phần dữ liệu mới rồi nối vào M: M(k+1) = [M(k) ; Encoder(dữ liệu mới)].

Đừng nhầm M với State Store: State Store là dữ liệu dạng bảng trong DB, lưu lâu dài, người đọc được; M là vector trong RAM khi Rex chạy, chỉ Rex hiểu, mất khi vụ xong.

6.2 Tầng nghĩ: Latent PonderBlock

Khối lặp T bước (2–4), dùng chung trọng số, có cổng kiểm soát:

```
h(t+1) = LayerNorm( h(t) + g(t) ⊙ ( CrossAttn(h(t), M) + MLP(h(t)) ) )
g(t)   = σ( W_g · h(t) + b_g )
```

- **CrossAttn(h, M):** h “đặt câu hỏi” cho M và lấy phần chứng cứ liên quan nhất, như lật lại hồ sơ tìm đoạn cần xem.
- **MLP(h)** (tài liệu Rex gọi là Cell): hiệu chỉnh tính từ chính h, như tự suy luận từ những gì đã biết.
- **Cộng rồi LayerNorm:** giữ cái đã hiểu, cộng phần mới, chuẩn hóa để số không phình to.

Vì h đổi sau mỗi bước, lần đọc sau tập trung vào phần chứng cứ khác. Vai trò mong muốn: bước 1 làm nổi chứng cứ mâu thuẫn, bước 2 đối chiếu dữ kiện then chốt còn thiếu, bước 3 dồn về một hành động. Model không bị ép đúng thứ tự này; phải chạy ablation mới biết mỗi bước đóng góp gì.

Ba cơ chế chống sụp đổ biểu diễn:

1. Cổng thiên về giữ nguyên trạng thái: b_g = −2,0 nên cổng ban đầu khoảng 0,12; W_g khởi tạo bằng 0.
2. LayerScale hệ số khởi tạo 1e-4 cho nhánh attention và MLP.
3. LayerNorm sau mỗi bước để giữ độ lớn vector ổn định qua các lần unroll.

```
class LatentPonderBlock(nn.Module):
    def __init__(self, d_model=768, nhead=12, dim_ff=2048):
        super().__init__()
        self.cross_attn = nn.MultiheadAttention(d_model, nhead,
batch_first=True)
        self.ln_attn = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(nn.Linear(d_model, dim_ff), nn.GELU(),
                                 nn.Linear(dim_ff, d_model))
        self.ln_out = nn.LayerNorm(d_model)
        self.gamma_attn = nn.Parameter(1e-4 * torch.ones(d_model))
        self.gamma_mlp = nn.Parameter(1e-4 * torch.ones(d_model))
        self.gate_proj = nn.Linear(d_model, d_model)
        nn.init.constant_(self.gate_proj.bias, -2.0)
        nn.init.zeros_(self.gate_proj.weight)

    def forward(self, h, M, memory_mask=None):
        # h: [B, 1, d_model], M: [B, L, d_model]
        attn_out, _ = self.cross_attn(h, M, M, key_padding_mask=memory_mask)
        h_inter = self.ln_attn(h + self.gamma_attn * attn_out)
        delta = self.gamma_mlp * self.mlp(h_inter)
        g = torch.sigmoid(self.gate_proj(h_inter))
        return self.ln_out(h + g * (attn_out + delta))
```

Lưu ý: dòng cuối cộng `attn_out` chưa nhân `gamma_attn`, nên LayerScale của nhánh attention chỉ tác động lên `h_inter` chứ không lên residual chính; muốn khớp mô tả thì dòng cuối nên là g * (self.gamma_attn * attn_out + delta).

6.3 Tầng chấm điểm: bốn head

Cả bốn head đọc cùng vector cuối h_T; mỗi head là một MLP nhỏ (Linear, GELU, Linear) trả lời đúng một câu hỏi.

Head	Trả lời câu hỏi	Đầu ra	Vai trò
Sufficiency	Đã đủ chứng cứ để được phép kết luận chưa?	$S \in [0, 1]$ (Sigmoid)	Ngưỡng cho phép kết luận: $S \geq 0{,}85$
Verdict	Nếu kết luận ngay thì nghiêng nhãn nào?	Logit trên tập nhãn đóng (CLEAN, FRAUD)	Chỉ được công nhận khi S đạt ngưỡng
Tool Matcher	Nếu chưa đủ thì tra thêm cái gì?	Vector chiếu 256 chiều, so với embedding mô tả tool	Chọn tool kế tiếp khi $S < 0{,}85$

Head	Trả lời câu hỏi	Đầu ra	Vai trò
Risk	Vụ có nguy hiểm đến mức phải giao người không?	$R \in [0, 1]$ (Sigmoid)	Vượt ngưỡng rủi ro nghiệp vụ thì chuyển người

S không phải độ tự tin. Độ tự tin nằm ở Verdict Head; S hỏi đã đủ dữ kiện để được chọn chưa. Hai số có thể lệch xa: hồ sơ mới có IP lạ, chưa tra thiết bị, Verdict có thể nghiêng FRAUD 90% nhưng S chỉ 0,4 vì thiếu dữ kiện then chốt, nên Rex đi tra thiết bị. Tách riêng S là để chặn lỗi kết luận vội từ một tín hiệu mạnh.

6.4 Lớp runtime

`Agent.predict()` hiện nhận một chuỗi trạng thái và từ điển câu hỏi, rồi chạy một lượt qua `DecisionModel`. Bản mới là `InvestigatorAgent`: giữ ngữ cảnh phiên, giữ tensor M và vector h qua các vòng ngoài, và trả về mã lệnh điều khiển (kết luận, gọi công cụ, chuyển người) thay vì chỉ một nhãn.

6.5 Phần cứng

h chỉ vài KB, M cỡ vài MB. Chạy bằng ONNX Runtime trên CPU, dưới 300 MB RAM.

7. Gọi công cụ bằng vector và MCP

Rex không sinh JSON để gọi công cụ: nó chọn công cụ bằng độ tương đồng vector, còn tham số do lớp điều phối lấy thẳng từ State Store.

7.1 Tool Registry và zero-shot matching

Mỗi công cụ có một mô tả bằng chữ; embedding của mô tả tính một lần lúc khởi động:

```
e_tool[i] = Normalize( Encoder( description[i] ) ),  thuộc R^d_proj
```

Tool Matcher chiếu h vào cùng không gian d_proj (256 chiều); công cụ có cosine similarity lớn nhất được chọn. Thêm công cụ mới chỉ cần viết mô tả, không train lại — nhưng điều này chỉ đúng khi model đã train với đủ nhiều tool có mô tả đa dạng.

7.2 Tham số tiền định hình

Orchestrator đọc schema của công cụ rồi lấy giá trị trực tiếp từ State Store. Cách này loại bỏ lỗi cú pháp JSON, sai tên trường và việc mô hình tự bịa tham số.

Công cụ	Trường State Store
`CALL_TOOL_IP_LOOKUP`	`state.network.session_ip`
`CALL_TOOL_DEVICE_LOOKUP`	`state.device.fingerprint_id`
Kiểm tra lịch sử tài khoản	`state.account.user_id`

7.3 Một lượt gọi công cụ: 3 bước gọi, 4 pha nạp chứng cứ

Gọi công cụ:

1. **Tool Matcher chọn công cụ:** chiếu h vào không gian embedding mô tả, lấy cosine lớn nhất (Top-K khi gọi song song).
2. **Orchestrator lấy tham số từ State Store** theo schema công cụ, ví dụ `state.network.session_ip`.
3. **Gọi MCP Server:** `ClientSession` của MCP qua Streamable HTTP hoặc Stdio, chạy bất đồng bộ bằng `asyncio.gather`.

Nạp chứng cứ tăng biên:

1. **Cập nhật State Store và trace:** ghi kết quả có cấu trúc vào nhóm `tool_results` và vào Trace Log phục vụ giải trình.
2. **Mã hóa tăng biên:** đóng gói kết quả thành chuỗi ngắn dưới 64 token, mã hóa trong 3–5 ms, không mã hóa lại hồ sơ gốc.
3. **Ghép bộ nhớ:** M(k+1) = [M(k) ; M_new] theo chiều chuỗi, mở rộng attention mask cho cross-attention.
4. **Chuyển trạng thái:** h_0 của vòng sau = h_T của vòng trước, nên model không quên mâu thuẫn đang cân nhắc.

Vòng suy ngẫm kế tiếp dùng CrossAttn(h, M mới) đối chiếu thông tin mới với giả thuyết cũ, nâng S hoặc nhắm truy vấn sâu hơn.

7.4 Gọi công cụ song song

Gọi song song giảm số vòng điều tra từ 3–5 xuống 1–2:

- **Ít vòng hơn:** các bước thu thập ban đầu (tra IP, tra thiết bị, lịch sử tài khoản) phần lớn độc lập, tiết kiệm 30–50% thời gian chờ.
- **Độ trễ băng công cụ chậm nhất:** max(latency_i) thay vì tổng. Dịch vụ nội bộ phản hồi 3–8 ms thì gọi 3 công cụ tốn dưới 10 ms mạng.
- **Xác minh chéo:** nhiều góc dữ liệu ngay vòng 1 giúp khối suy ngẫm triệt tiêu nhiễu.

Chọn băng Sigmoid + Top-K giảm dần. Tool Matcher tính xác suất độc lập p[i] = σ(z[i]) cho từng công cụ, rồi giữ K_t công cụ cao nhất. Ngân sách giảm theo vòng: vòng 1 K = 3 (khám phá rộng), vòng 2 K = 2 (tập trung), từ vòng 3 K = 1 (xác nhận). Công cụ chỉ chạy khi vừa thuộc Top-K_t vừa có p[i] ⩾ p_min; nếu không công cụ nào qua ngưỡng thì vẫn chọn Top-1. Giá trị p_min chưa xác định.

Rủi ro	Mô tả	Biện pháp
Bão truy vấn	Chuẩn của h biến động sau các chu kỳ; 5–10 công cụ cùng vượt 0,5, làm tắc connection pool của DB nội bộ	Top-K giới hạn cứng số công cụ mỗi vòng
Đóng băng	Model phân vân, mọi σ(z_i) rơi vào 0,2–0,4, không công cụ nào qua ngưỡng trong khi S < 0,85	Luôn chọn Top-1 khi không công cụ nào qua p_min
Thiếu cạnh tranh	Chấm độc lập nên không tự loại công cụ thừa, ví dụ đã có hồ sơ định danh đầy đủ thì tra riêng số điện thoại là dư	Chưa có cơ chế loại trừ lẫn nhau; theo dõi khi thử nghiệm

8. Suy luận giả định và loại trừ (POE)

Rex hoạt động như một thám tử: coi cảnh báo là quan sát dị thường O, đặt tập giả thuyết H, rồi loại dần bằng chứng cứ thực tế bằng mặt nạ logit trong không gian vector.

Cơ sở. Suy luận giả định (Peirce) đi từ O ngược về giả thuyết giải thích tốt nhất. Theo bản phân tích, nghiên cứu POE (Ma và Du 2023) cho thấy tách hai pha — loại phương án bất hợp lý rồi mới phán quyết trên phần còn lại — cho độ chính xác cao hơn chọn trực tiếp. LLM sinh văn bản khó tự loại trừ bằng ngôn ngữ tự nhiên vì giải thích vì sao một phương án sai dễ gây ảo giác, nên thiết kế chuyển việc này sang mặt nạ logit.

Ví dụ tập giả thuyết (bài toán điều tra gian lận):

Mã	Giả thuyết
H1	Chiếm đoạt tài khoản trái phép (ATO) qua đánh cắp phiên
H2	Gian lận thân thiện: người thân trong gia đình sử dụng
H3	Khách hàng đổi hành vi hợp lệ (đi du lịch, mua sắm đột xuất)
H4	Tấn công mạng tự động (botnet, giả lập thiết bị)

8.1 Chấm điểm và mặt nạ loại trừ

```
Score(H_j) = FFN( CrossAttn( e_Hj, M ) )
M_elim[j]  = 1  nếu Score(H_j) ≥ θ_elim
M_elim[j]  = 0  nếu Score(H_j) < θ_elim
```

```
Logits_final = Logits_raw + (M_elim − 1) × ∞
```

M_elim = 0 nghĩa là giả thuyết bị bác bỏ hoàn toàn, ví dụ thiết bị có chứng chỉ TPM hợp lệ đăng ký 2 năm mâu thuẫn với H4. Verdict Head chỉ được phán quyết trên các kịch bản chưa bị bác bỏ.

8.2 Chọn công cụ theo độ tăng thông tin

Tool Matcher ưu tiên công cụ làm giảm entropy kỳ vọng lớn nhất trên các giả thuyết còn sống (M_elim[j] = 1):

```
P_t[j] = σ( Score(H_j) ),  với j còn sống (M_elim[j] = 1)
H(P_t) = − Σ_j [ P_t[j] · log P_t[j] + (1 − P_t[j]) · log(1 − P_t[j]) ]
```

```
T* = argmax_{T_k} E[ H(P_t) − H(P_(t+1) | Observation(T_k)) ]
```

Ví dụ: nếu chỉ còn phân vân giữa H1 và H3, model bỏ qua tra số dư hay tra thẻ tín dụng, gọi thẳng công cụ kiểm tra lịch sử chuyến bay hoặc thông báo xác thực hai yếu tố, vì chỉ hai công cụ này phân định được hai giả thuyết còn lại.

8.3 Vòng loại trừ

1. Nhận cảnh báo ban đầu O.
2. Khởi tạo tập giả thuyết H; mỗi H_j là vector e_Hj, ban đầu đều mở (M_elim[j] = 1).

3. Chấm điểm từng giả thuyết; giả thuyết nào có Score < θ_elim thì loại (M_elim[j] = 0).
4. Chọn công cụ T* cực đại hóa kỳ vọng giảm entropy của giả thuyết còn sống.
5. Gọi công cụ, nối kết quả vào M, quay lại bước 3.
6. Khi S ⩾ 0,85: phán quyết trên giả thuyết còn sống bằng Logits_final.

Tập nhãn của Verdict Head trong tài liệu Rex là CLEAN/FRAUD, còn POE dùng H1–H4; cần chốt cách hai tầng này ghép với nhau.

9. Lớp điều phối và an toàn

Rex chỉ đưa điểm số và đề xuất; lớp điều phối (code thường bao quanh Rex) thực thi tool, giữ giới hạn an toàn và bảo đảm vụ nào cũng dừng, hành động nặng nào cũng qua người.

Bốn nguyên tắc vận hành:

- **Không có luật viết tay.** Rex học từ dữ liệu rằng thiếu dữ kiện then chốt thì S thấp, và tool lấp được chỗ thiếu sẽ được điểm cao.
- **Đầu ra thuộc tập nhãn đóng:** mỗi vòng đúng một nhãn `CALL_TOOL_*` hoặc `VERDICT_*`; không sinh văn bản nên không phải parse.
- **Thời gian mỗi vòng gần cố định,** không phải chuỗi sinh chữ dài ngắn tùy lúc.
- **Model chọn, lớp điều phối giới hạn.** Rex tự quyết gọi tool nào (kể cả gọi lại), lớp ngoài giữ giới hạn.

9.1 Phân quyền tool theo mức tác động

Mức	Ví dụ	Rex được làm gì	Giới hạn
Đọc dữ liệu	Tra IP, tra thiết bị, xem lịch sử giao dịch	Tự gọi, kể cả gọi lại	Tính vào số vòng và điều kiện bế tắc
Tác động nhẹ	Ảnh hưởng nhỏ tới khách, đảo ngược được	Tự gọi	Hạn mức theo vụ và theo người dùng; có idempotency key
Tác động nặng	Ảnh hưởng lớn hoặc khó đảo ngược	Chỉ được đề xuất, sau đó vụ chuyển người	Người tiếp quản toàn bộ vụ; có idempotency key

Hạn mức tính theo người dùng vì nhiều cảnh báo về cùng tài khoản có thể sinh nhiều vụ chạy song song. Idempotency key bảo đảm lệnh bị gửi lại do lỗi mạng không bị thực thi hai lần.

9.2 Bốn điều kiện chuyển người

Chỉ kết luận tự động khi cùng lúc S ⩾ 0,85 và vụ không thuộc nhóm rủi ro cao. Sau mỗi vòng, lớp điều phối kiểm tra theo thứ tự:

1. **Rủi ro cao (`HIGH_RISK`):** chạm sàn luật rủi ro hoặc Risk Head vượt ngưỡng R. Rex vẫn thu thập chứng cứ, nhưng khi đủ thì người kết luận.
2. **Bế tắc (`DEADLOCK`):** S tăng ít hơn ε trong 2 vòng liên tiếp.
3. **Hết ngân sách (`BUDGET_EXHAUSTED`):** đã gọi tool đủ N_max lần.
4. **Tool bị chặn (`TOOL_BLOCKED`):** Rex đề xuất tool tác động nặng, hoặc tool tác động nhẹ đã hết hạn mức.

Người duyệt nhận đủ vết điều tra: tool đã gọi kèm tham số, S từng vòng, và hành động Rex đề xuất nếu có.

9.3 Chống lặp vô ích

Không có luật cứng cấm gọi trùng; thay vào đó có bốn lớp bảo vệ:

1. Rex thấy lịch sử của chính nó: bản ghi các lần gọi tool năm trong M.
2. Khi train, gọi thừa bị phạt.
3. Lặp vô ích tự dẫn tới dừng: gọi không thêm thông tin thì S không tăng; hai vòng như vậy kích hoạt bế tắc.
4. N_max là lớp cuối: kể cả khi mọi cơ chế khác hỏng, vụ vẫn dừng sau tối đa N_max lần gọi.

9.4 Các ngưỡng cần chốt từ dữ liệu

Tham số	Ý nghĩa	Cách xác định
ε	Mức tăng S tối thiểu để coi là có tiến triển	Phân phối mức tăng S mỗi vòng trên tập kiểm định
N_max	Số lần gọi tool tối đa mỗi vụ	p99 số lần gọi của Teacher, cộng khoảng dư
R	Ngưỡng risk bắt buộc có người duyệt	Chính sách rủi ro nghiệp vụ
Sàn luật rủi ro	Điều kiện luôn chuyển người, bất kể Risk Head	Chính sách rủi ro nghiệp vụ
p_min	Xác suất tối thiểu để một tool được chạy song song	Chưa xác định

Tham số	Ý nghĩa	Cách xác định
θ_elim	Ngưỡng loại giả thuyết	Chưa xác định
Hạn mức tool nhẹ	Số lần gọi mỗi vụ và mỗi người dùng trong một khoảng thời gian	Chính sách sản phẩm
Timeout mỗi tool	Thời gian chờ tối đa cho một MCP server	SLA từng hệ thống nguồn

10. State Store và Decision Adapter

State Store giữ hồ sơ vụ ở đầu vào của Rex; Decision Adapter ghép kết luận với hồ sơ ở đầu ra. Cả hai chưa được tài liệu Rex gốc định nghĩa, nên nội dung dưới đây là đề xuất **[Cần xác nhận]**.

10.1 Mở vụ và State Store

Rex không tự tìm việc: khi một sự kiện (giao dịch, đăng nhập) chạm cảnh báo, hệ thống mở vụ tạo ticket, ghi hồ sơ ban đầu, rồi mới giao Rex điều tra. Nếu dự án đã có hệ thống quản lý case thì đó chính là State Store, chỉ cần kiểm tra đủ các trường.

Nhóm	Trường ví dụ	Ghi lúc nào	Ai ghi
Định danh	`ticket_id`, `user_id`, `created_at`	Lúc mở vụ	Hệ thống mở vụ
Sự kiện gốc	`action_type`, `amount`, `alert_reason_code`, thiết bị và IP của phiên	Lúc mở vụ	Hệ thống mở vụ
Kết quả tra cứu	`tool_results`: tên tool, tham số, dữ liệu trả về, thời điểm	Sau mỗi lần gọi tool	Lớp điều phối
Trạng thái điều tra	`status` (đang điều tra, đã kết luận, đã chuyển người), số lần gọi tool, S từng vòng	Sau mỗi vòng	Lớp điều phối
Kết luận	`decision_seq`, `final_action`, `decision_source`, `decided_at`	Khi Rex hoặc người chốt	Lớp điều phối, giao diện duyệt

Kết quả tool phải được ghi ngược vào State Store dạng có cấu trúc (nhóm “Kết quả tra cứu”), nếu không tool sau không dùng được dữ liệu tool trước tìm ra — ví dụ log đăng nhập ra `device_id` thì tool tra thiết bị cần lấy được.

10.2 Decision Adapter

Đoạn code nhỏ, không có AI (có thể là một hàm trong lớp điều phối), ghép kết quả vụ với hồ sơ thành một message rồi đẩy vào queue. **[Cần xác nhận: team nào viết và quản lý.]**

Vì sao không đẩy thẳng nhãn của Rex vào queue:

1. **Rex chỉ biết kết luận.** `VERDICT_FRAUD` không cho biết vụ của ai, mã ticket, loại giao dịch, thời điểm.
2. **Nhãn của Rex không phải lúc nào cũng là quyết định cuối.** Vụ rủi ro cao dù Rex ra FRAUD vẫn chuyển người; Adapter gom hai nguồn về cùng format.
3. **Chặn dữ liệu nội bộ.** Message chỉ mang con trỏ `trace_id`; Report Service đọc trace từ kho có kiểm soát quyền.
4. **Tách Rex khỏi phía sau.** Đổi nhãn hoặc thay model chỉ cần sửa Adapter.

Đầu vào — kết quả vụ từ lớp điều phối (đề xuất):

```
{
  "ticket_id": "TCK-2026-88712",
  "outcome": "AUTO_VERDICT",
  "verdict": "VERDICT_FRAUD",
  "escalation_reason": null,
  "decided_at": "2026-10-10T22:58:05Z",
  "trace_id": "rex-trace-7f3a"
}
```

- `outcome`: `AUTO_VERDICT` (Rex tự kết luận) hoặc `ESCALATED` (chuyển người).
- `escalation_reason`: `HIGH_RISK`, `DEADLOCK`, `BUDGET_EXHAUSTED` hoặc `TOOL_BLOCKED` (mục 9.2).
- `trace_id`: trỏ tới vết điều tra đầy đủ; Adapter chép vào message.

Cách map. Adapter không bao giờ đọc S để quyết định, chỉ đọc `outcome`.

Kết quả vụ	final_action	decision_source
Tự kết luận `VERDICT_CLEAN`	ALLOW	REX_AUTO
Tự kết luận `VERDICT_FRAUD`	BLOCK	REX_AUTO
Chuyển người, mọi lý do	ESCALATE_HUMAN	REX_AUTO
Người duyệt chốt	ALLOW hoặc BLOCK	HUMAN_REVIEW

Khi người duyệt chốt, Adapter dùng lại cùng logic với 3 thay đổi: `final_action` lấy từ kết luận của người, `decision_source = "HUMAN_REVIEW"`, và `supersedes_request_id` trỏ về message trước.

10.3 Vụ chuyển người sinh hai message

Lần	Khi nào	request_id	final_action	Khách nhận
1	Ngay khi Rex chuyển vụ	TCK-2026-77001:1	ESCALATE_HUMAN	“Đang xem xét”, chỉ template
2	Khi người duyệt chốt	TCK-2026-77001:2	ALLOW hoặc BLOCK	Báo cáo chính thức, thay cho lần 1

`request_id` có dạng `{ticket_id}:{decision_seq}`, nên mỗi lần quyết định có khóa chống trùng riêng mà vẫn truy được về cùng một vụ. Khi vụ đã chuyển người, message không mang nhãn Rex đang nghiêng, để báo cáo không dựa vào nhận định chưa được chốt.

11. Pipeline lập báo cáo

Nửa sau nhận một message cho mỗi quyết định, dựng đúng một báo cáo điều tra từ trace (code điền dữ kiện, DeepSeek chỉ viết diễn giải), và bảo đảm báo cáo lẫn thông báo tới nơi nhờ lease và transactional outbox.

11.1 Message Decision_Finalized

Đây là thứ duy nhất đi từ nửa quyết định sang nửa giải thích. Message chỉ mang con trỏ, không mang vết điều tra: S, logits, risk, danh sách tool và kết quả tra cứu nằm trong kho trace, Report Service đọc theo `trace_id`. Trường `mcp_tool_history` của bản trước đã bỏ vì chỉ là bản sao không đầy đủ của trace.

```
{
  "request_id": "TCK-2026-88712:1",
  "user_id": "usr_9988",
  "ticket_id": "TCK-2026-88712",
  "trace_id": "rex-trace-7f3a",
  "decision_seq": 1,
  "decision_source": "REX_AUTO",
  "supersedes_request_id": null,
  "final_action": "BLOCK",
  "action_type": "WITHDRAWAL",
  "user_reason_code": "UNUSUAL_GEO_LOCATION",
  "event_timestamps": {
    "detected_at": "2026-10-10T22:58:00Z",
    "decided_at": "2026-10-10T22:58:05Z",
    "applied_at": "2026-10-10T22:58:06Z"
  },
  "masked_metadata": { "amount_bucket": "10–100 triệu VND" }
}
```

Trường	Ý nghĩa	Ai dùng phía sau
`request_id`	Khóa chống trùng: cùng request_id là cùng một quyết định, chỉ sinh một báo cáo	Report Service (kiểm tra DB)
`user_id`	Vụ của ai, gửi thông báo cho ai	Báo cáo (case), thông báo khách
`ticket_id`	Mã vụ, giữ nguyên qua mọi lần quyết định	Báo cáo (meta), thông báo khách
`trace_id`	Con trỏ tới vết điều tra đầy đủ của Rex	Report Service
`decision_seq`	Lần quyết định thứ mấy: 1 là Rex, 2 là người	Dashboard, hiển thị bản mới nhất
`decision_source`	REX_AUTO hoặc HUMAN_REVIEW	Báo cáo, audit
`supersedes_request_id`	Báo cáo này thay cho báo cáo nào	Dashboard
`final_action`	ALLOW, BLOCK hoặc ESCALATE_HUMAN; báo cáo không bao giờ được nói ngược	Báo cáo, kiểm duyệt, chọn template

Trường	Ý nghĩa	Ai dùng phía sau
`action_type`	Rút tiền, đăng nhập, chuyển khoản…	Báo cáo (case), template
`user_reason_code`	Mã lý do mở vụ	Báo cáo (case.trigger_rule), template
`event_timestamps`	Mốc phát hiện, ra quyết định, áp dụng	Báo cáo
`masked_metadata`	Dữ liệu phụ đã che PII, chỉ dùng cho thông báo khách	Template thông báo

Queue tự gắn thêm thông tin quản lý (ví dụ số lần giao); Report Service dùng số này để biết khi nào chuyển vào DLQ.

11.2 Report Service: lease và heartbeat

Service chạy nhiều worker song song; mỗi worker lặp: lấy message, xử lý, ACK, lấy tiếp. Luồng một message: **A.** giành lease → **B.** đọc trace → **C.** soạn prompt → **D.** DeepSeek viết analysis → **E.** kiểm duyệt → **F.** lưu báo cáo + outbox trong một transaction rồi mới ACK (SUCCESS nếu đạt, FALLBACK nếu không).

Queue chỉ bảo đảm giao ít nhất một lần, nên một message có thể đến hai lần:

Tình huống	Hai lần giao	Đủ để chặn
Worker làm xong nhưng chết trước khi ACK	Lần lượt	Kiểm tra DB thấy SUCCESS
Bên gửi gửi trùng do không nhận được xác nhận	Lần lượt	Kiểm tra DB thấy SUCCESS
AI trả lời chậm hơn thời gian queue chịu chờ ACK	Cùng lúc	Cần lease
Deploy hoặc scale pod làm queue chia lại message	Cùng lúc	Cần lease

Chỉ đánh dấu PROCESSING thì chặn được trùng nhưng sinh lỗi khác: worker giữ cờ mà chết thì cờ nằm im mãi. **Lease** là cờ có hạn dùng 60 giây: còn hạn thì không ai được đụng, hết hạn thì người khác được lấy lại.

- **Heartbeat:** worker còn sống thì cứ 15 giây gia hạn lease, kiểm tra đúng `lease_id` của mình. Gia hạn thất bại nghĩa là đã bị giành việc: worker hủy lời gọi AI và dừng ngay.
- **Message trùng khi lease còn hạn:** NACK delay 30 giây. Quá 3 lần NACK thì chuyển DLQ và báo vận hành.
- **Lease đã hết hạn:** worker mới giành lease bằng một UPDATE nguyên tử rồi chạy thẳng fallback.
- **Job quét mồ côi:** mỗi phút quét các bản ghi PROCESSING hết lease mà không có message quay lại, xử lý như trên.

11.3 Báo cáo điều tra (6 phần)

Phần lớn báo cáo do code điền thẳng từ trace; LLM chỉ viết `analysis`. `meta` (ID báo cáo, phiên bản model, trạng thái sinh) và `case` (user, loại giao dịch, số tiền, rule mở vụ) lấy từ message và State Store.

decision

Trường	Kiểu	Nguồn	Ví dụ
final_action	ALLOW / BLOCK / ESCALATE_HUMAN	Message	BLOCK
decision_source	REX_AUTO / HUMAN_REVIEW	Message	REX_AUTO
outcome	AUTO_VERDICT / ESCALATED	Trace	AUTO_VERDICT
verdict	VERDICT_* hoặc null	Trace	VERDICT_FRAUD
verdict_probs	{CLEAN, FRAUD}	Trace, softmax của logits	{0.12, 0.88}
final_sufficiency	number	Trace, S vòng cuối	0.91
sufficiency_threshold	number	Cấu hình	0.85
risk_score / risk_threshold	number / number	Trace / cấu hình	0.30 / 0.70
escalation_reason	HIGH_RISK / DEADLOCK / BUDGET_EXHAUSTED / TOOL_BLOCKED hoặc null	Trace	null

Trường	Kiểu	Nguồn	Ví dụ
decided_at	datetime	Trace	2026-10-10T22:58:05Z

steps[] (phần chính)

Trường	Kiểu	Nguồn	Ví dụ
step_no	int	Trace	1
action	CALL_TOOL_* / VERDICT_*	Trace	CALL_TOOL_IP_LOOKUP
tool_impact	READ / LIGHT / HEAVY	Registry	READ
params	object, đã che PII	Trace	{"ip": "103.x.x.21"}
result	object có cấu trúc	Trace (kết quả tool)	{"country": "SG", "seen_before": false}
sufficiency_before / after	number	Trace	0.40 / 0.62
top_tool_scores	top 3 {tool, score}	Trace	Rex đã cân nhắc tool nào
duration_ms	int	Lớp điều phối	48
status	OK / TIMEOUT / ERROR / BLOCKED	Lớp điều phối	OK

analysis (DeepSeek viết)

Trường	Kiểu	Yêu cầu
summary	string, 2–3 câu	Tóm tắt vụ và kết luận
key_findings[]	{text, cited_steps[]}	Mỗi phát hiện trích ít nhất một step_no
reasoning	string	Vì sao chuỗi chứng cứ dẫn tới kết luận, có trích bước
open_questions[]	string	Điểm còn mơ hồ hoặc chưa tra

Trường	Kiểu	Yêu cầu
suggested_next_actions[]	string	Cho người duyệt, chỉ có khi ESCALATE_HUMAN

human_review (điền khi người duyệt chốt)

Trường	Kiểu	Ý nghĩa
reviewer_id	string	Ai duyệt
reviewed_at	datetime	Duyệt lúc nào
final_action	ALLOW / BLOCK	Kết luận của người
agrees_with_rex	bool	Có đồng ý với nhãn Rex đang nghiêng không
note	string	Ghi chú
missed_tools[]	string	Tool lẽ ra nên gọi; cũng là nhãn để train lại Rex

`analysis` là diễn giải của LLM dựa trên trace, không phải lý do thật của Rex (Rex chỉ ra con số, không giải thích được). Dashboard phải ghi rõ "Diễn giải tự động" ở phần này.

11.4 Vết điều tra Rex phải ghi

Muốn điền đủ báo cáo, lớp điều phối phải ghi thêm vào trace một số trường **[Cần xác nhận với team Rex]**. Kho trace cần giữ đủ lâu để audit và phải phân quyền xem vì chứa dữ liệu nhạy cảm.

Trường trong trace	Tài liệu Rex đã có chưa	Dùng cho
Tool đã gọi, tham số	Có	steps[].action, params
S theo từng vòng	Có	sufficiency_before, sufficiency_after
Hành động Rex đề xuất	Có	steps[].action, escalation_reason
Kết quả tool dạng có cấu trúc	Chưa	steps[].result
Điểm các tool ở mỗi vòng	Chưa	top_tool_scores
verdict_logits và risk ở vòng cuối	Chưa	verdict_probs, risk_score

Trường trong trace	Tài liệu Rex đã có chưa	Dùng cho
Thời gian và trạng thái từng lần gọi	Chưa	duration_ms, status
Phiên bản model	Chưa	rex_model_version

11.5 Prompt và gọi DeepSeek

- **Đầu vào:** case, decision, steps đã che PII. Không đưa dữ liệu ngoài trace vào prompt.
- **Yêu cầu trong prompt:** chỉ dùng dữ kiện có trong đầu vào; mỗi phát hiện ghi `cited_steps`; không kết luận khác `final_action`.
- **Chống prompt injection:** kết quả tool có thể chứa chuỗi do người dùng đặt (tên thiết bị, nội dung chuyển khoản); bọc riêng như dữ liệu và giới hạn độ dài.
- **Model:** `deepseek-chat` (DeepSeek-V3) ở JSON Mode, timeout 20 giây. Không dùng R1. Nên pin phiên bản thay vì dùng alias.
- Heartbeat chạy nền song song với lời gọi và dừng khi lời gọi xong. Môi trường dev dùng mock JSON thay lời gọi DeepSeek.

11.6 Kiểm duyệt và fallback

Lớp "giấu tín hiệu gian lận" của bản trước đã bỏ khỏi báo cáo vì admin cần thấy chính các tín hiệu đó; thay bằng phân quyền xem trên dashboard. Việc giấu tín hiệu chỉ còn áp dụng cho thông báo gửi khách.

Lớp	Kiểm tra gì	Cách làm
1. Schema	Đủ trường, đúng kiểu	Khớp JSON schema
2. Không bịa	Mọi cited_steps tồn tại; mọi con số, quốc gia, thời điểm trong analysis khớp steps hoặc decision	Code trích giá trị trong văn bản rồi so với trace
3. Không mâu thuẫn	summary và reasoning không nói ngược final_action	LLM tự khai thêm trường outcome, code so với final_action
4. Che PII	Không lộ số tài khoản, số điện thoại, IP đầy đủ…	Quét mẫu PII trên params, result và analysis

Fallback. Khi kiểm duyệt fail, DeepSeek quá 20 giây, hoặc lease hết hạn phải giành lại, báo cáo vẫn lưu với `generation_status = FALLBACK`. meta, case, decision, steps vẫn đầy đủ vì do code điền; chỉ `analysis` là null và dashboard hiển thị "Không có diễn giải tự động".

11.7 Thông báo cho khách

Chỉ dùng template, không gọi LLM. Template chọn theo bộ (`final_action` , `action_type` , `user_reason_code`); mã lý do chưa có template thì dùng template mặc định. Thông báo không bao giờ nêu giá trị tín hiệu (quốc gia, IP, thiết bị) vì chi tiết đó chỉ giúp kẻ gian biết cần né gì. Template do team Rủi ro và CSKH duyệt.

final_action	Nội dung	Ví dụ
BLOCK	Giao dịch bị tạm dừng, liên hệ CSKH kèm mã tra cứu	“Giao dịch rút tiền tạm thời bị chặn. Vui lòng liên hệ CSKH với mã TCK-2026-88712.”
ESCALATE_HUMAN	Giao dịch đang được xem xét	“Giao dịch của bạn đang được kiểm tra thêm. Chúng tôi sẽ thông báo khi có kết quả.”
ALLOW	Thường không cần gửi [Cần xác nhận]	—

11.8 Outbox và gửi

“Sinh xong” và “gửi xong” là hai việc khác nhau. Worker ghi báo cáo và dòng “chờ gửi” vào `report_outbox` trong cùng một transaction; việc gửi do Outbox Worker riêng phụ trách.

Mỗi request ghi hai dòng outbox, retry độc lập: `channel = ADMIN_DASHBOARD` mang báo cáo điều tra, `channel = CUSTOMER_NOTIFY` mang thông báo template. Bảng cần thêm cột `channel` và ràng buộc `UNIQUE(request_id, channel)` . Với vụ ALLOW, nếu team chốt không báo khách thì chỉ ghi dòng thứ nhất.

```
BEGIN;
UPDATE report_jobs
   SET generation_status = 'SUCCESS',   -- hoặc 'FALLBACK'
       content = :final_report_json, updated_at = now()
 WHERE request_id = :rid AND lease_id = :my_lease;
-- 0 dòng = đã mất lease → ROLLBACK, không gửi gì
INSERT INTO report_outbox (request_id, user_id, payload, delivery_status,
       attempts, next_retry_at, retry_window_started_at, created_at)
VALUES (:rid, :uid, :final_report_json, 'PENDING', 0, now(), now(), now());
COMMIT;
-- Sau khi commit mới ACK message gốc
```

`delivery_status` (PENDING, SENT, FAILED) chỉ năm ở `report_outbox` . Message trùng đến mà thấy báo cáo đã SUCCESS hoặc FALLBACK thì worker chỉ cần ACK.

Nhiều Outbox Worker mà không gửi trùng. Worker không giữ lock lúc gọi HTTP: nó giành dòng có thời hạn bằng một UPDATE ngắn, commit ngay, rồi mới gửi. Mỗi lần lấy 5 dòng, mỗi lần gửi timeout 5 giây, tối đa 25 giây — vẫn nhỏ hơn hạn giành 60 giây. Outbox chỉ bảo đảm gửi ít nhất một lần, nên dashboard và app khách phải loại trùng theo `request_id`.

Kết quả gửi	Xử lý
Bên nhận xác nhận (200 OK)	`delivery_status = 'SENT'`
Lỗi, còn trong cửa sổ 24 giờ	Giữ PENDING; lần thử sau cách min(2^attempts phút, 2 giờ), khoảng 18 lần trong 24 giờ
Quá 24 giờ, hoặc đã thử 30 lần	FAILED, bắn cảnh báo `outbox_delivery_failed`

Dòng FAILED không tự gửi lại để tránh dồn dập vào endpoint đang hỏng. Sau khi sửa, vận hành replay thủ công: đặt lại PENDING, `attempts = 0`, `retry_window_started_at = now()`, ghi lại ai replay, lúc nào. Cửa sổ 24 giờ tính từ `retry_window_started_at` chứ không từ `created_at`, nếu không dòng vừa replay sẽ lập tức bị đánh FAILED lại.